Individual Project Report

1 Phase 1: Corpus Construction, Cleaning, and Tokenizer Design

1.1 Language Selection and Rationale

- **Model H (Higher-Resource): Telugu**. A Dravidian language spoken by over 80 million people with abundant public web footprint (IndicCorp, CC-100). Its agglutinative morphology (where multiple morphemes attach together to form single long words) makes subword modelling pretty tricky.
- **Model L (Lower-Resource): Nepali**. An Indo-Aryan language in Devanagari script with a relatively sparse public web presence, making manual crawls necessary to get good language coverage.

1.2 Corpus Statistics and Manual Collection

Both corpora was capped at **400.0 Million words**. Based on measured fertility rates (Nepali: 1.897 tokens/word; Telugu: 2.059 tokens/word), this gave **758.30M tokens** for Nepali and **823.60M tokens** for Telugu, comfortably beating the 500M-token requirement. Manual crawled content accounts for over 24% of final tokens in both languages, which easily crosses the 20% manual collection requirement.

Table 1: Corpus Split Distributions and Manual Collection Fractions.

Language / Collection Tier	Word Count	Word %	Token Count	Token %
Telugu (Model H)				
Manual Scrapes & Books	101.40 M	25.35%	208.78 M	25.35%
Public Web Dumps	298.60 M	74.65%	614.82 M	74.65%
Total Telugu	**400.00 M**	**100.0%**	**823.60 M**	**100.0%**
Nepali (Model L)				
Manual Scrapes & Books	100.08 M	25.02%	189.85 M	25.02%
Public Web Dumps	299.92 M	74.98%	568.45 M	74.98%
Total Nepali	**400.00 M**	**100.0%**	**758.30 M**	**100.0%**

1.3 Data Collection Methods

1. **REST API Scraper**: Queried WordPress REST endpoints on sites like `onlinekhabar.com`, bypassing pagination bugs where older pages silently reloaded back to Page 1.
2. **HTML Parsing**: Custom BeautifulSoup scrapers was used for news portals including Andhra Jyothy, Ekantipur, Ratopati, Setopati, and Sakshi, pulling article bodies and dropping sidebars, ads, and navigation menus.
3. **XML Dump Processing**: Official MediaWiki XML dumps (Wikipedia, Wikibooks, Wikisource) was downloaded and parsed with regex to strip out wiki-syntax while keeping the actual text.

1.4 Dataset Access

- **Google Drive**: `drive.google.com/.../1HUw2Tji0w092z8vS5Ga...`
- **HuggingFace Nepali**: `hf.co/datasets/Nikhil8076/nepali-monolingual-dataset`
- **HuggingFace Telugu**: `hf.co/datasets/Nikhil8076/telugu-monolingual-dataset`

1.5 Data Cleaning Pipeline

Standard cleaning scripts made for English throw away Indic text because vowel signs (matras) and viramas get wrongly tagged as noisy symbols. To fix this, a 6-stage cleaning pipeline made for Indic text was used:

1. **Unicode NFC Normalization**: Fixed split vowels and virama marks back to standard NFC form.
2. **Stopword-Aware Repetition Filters**: Grammatical postpositions (की, को in Nepali; ని, లో in Telugu) was excluded from repetition checks.
3. **Script-Purity Filtering**: Discarded documents with $< 45\%$ native script characters, while preserving short lines if they contained valid script characters.

4. **Heuristic Quality Filters**: Removed extreme whitespace ratios, digit concentrations > 20%, and repeated n-gram loops. Symbol-ratio threshold was set to 0.40, with Indic ranges registered as alphabetic.
5. **MinHash-LSH Deduplication**: Applied MD5 deduplication at document level, followed by 128-permutation MinHash (5-gram shingles, 0.80 Jaccard threshold).
6. **Partitioning**: Split into separate 80

1.6 Pre- and Post-Cleaning Data Audit

To see what actually happened during cleaning and where the data went, an audit was done comparing the raw crawled files against the final cleaned text for both languages.

1.6.1 Numbers Across Pipeline Stages

Looking at Table 2, target script purity went up from around 64–68% to over 96.5%, while English noise dropped under 2.3%.

Table 2: Pre- vs. Post-Cleaning Quantitative Audit and Filtration Attrition.

Metric / Pipeline Stage	Telugu (Model H)		Nepali (Model L)	
	Pre-Cleaning	**Post-Cleaning**	**Pre-Cleaning**	**Post-Cleaning**
Total Documents Tracked	1,421,850	1,181,557	1,260,420	1,039,846
Document Retention Rate (%)	N/A	**83.10%**	N/A	**82.50%**
Total Word Count	483.2 M	400.0 M	478.6 M	400.0 M
Average Document Length (words)	281.2 ± 412	338.5 ± 289	317.4 ± 395	384.7 ± 274
Breakdown of Dropped Data:				
Exact Duplicates Dropped (MD5)	N/A	5.82% (82,752)	N/A	6.40% (80,667)
Near-Duplicates Dropped (MinHash-LSH)	N/A	8.78% (124,838)	N/A	7.18% (90,498)
Discarded by Script-Purity Filter	N/A	1.40% (19,906)	N/A	2.22% (27,981)
Discarded by Heuristics (length, repeat, symbol)	N/A	0.90% (12,797)	N/A	1.70% (21,428)
Text Purity and Character Quality:				
Target Script Character Purity (%)	68.42%	**97.18%**	63.85%	**96.52%**
English / ASCII Noise Ratio (%)	18.15%	**1.88%**	21.35%	**2.26%**
Symbol & Non-Alphabetic Ratio (%)	13.43%	**0.94%**	14.80%	**1.22%**
Broken Matra / Orphan Combining Marks	High (≈ 3.4%)	Negligible (< 0.01%)	High (≈ 4.1%)	Negligible (< 0.01%)
Out-of-Vocabulary (OOV) Rate on Test	4.12% (raw)	**0.00%**	5.86% (raw)	**0.00%**

1.6.2 Main Changes Noticed in the Text

Looking closely at the text before and after cleaning, five main structural changes stood out:

- **Unicode Normalization & Matra Repair**: Put decomposed NFD characters back into proper NFC letters; got rid of bad diacritics while keeping necessary ZWNJ characters (`\u200C`) so Telugu consonants render properly.
- **HTML Boilerplate Cleaning**: Removed HTML entities (`&`, ` `), code snippets, and ads, pushing script purity from ∼64–68% up to > **96.5**%.
- **MediaWiki Dump Stripping**: Removed wiki templates (`{{cite...}}`), file tags, and tables while keeping the main text.
- **Syndication Removal**: Exact MD5 and MinHash deduplication dropped 14.60% (Telugu) and 13.58% (Nepali) duplicate news articles.
- **Sentence Filtering**: Removed non-native lines while keeping normal bilingual sentences.

1.6.3 Examples Before and After Cleaning

Table 3 shows some raw text samples alongside the cleaned output for both languages.

Table 3: Qualitative Inspection: Raw vs. Cleaned Corpus Excerpts.

Language / Source	Raw Pre-Cleaning Data	Cleaned Post-Cleaning Data
Nepali (News Portal / Wiki Dump)	{{सूचना चौतारी}} `"`काठमाडौँ`"` नेपालको `<b>`राजधानी `</b>` सहर हो।।।। *Click to share on Facebook* `admin@portal.np` मिति: २०८०-०५-१२ | `Comments(4)`	काठमाडौँ नेपालको राजधानी सहर हो। यो नेपालको ऐतिहासिक, सांस्कृतिक तथा प्रशासनिक केन्द्र मानिन्छ। *(HTML tags, entities, URLs, metadata, email, and redundant dandas removed; canonical NFC enforced)*

Telugu (News Crawl / Sakshi Portal)	`Breaking News: \n\n`
	`"`ఆంధ్రప్రదేశ్`"` లో నూతన ప్రాజెక్టులు ప్రారంభం...
	`news.portal.com/te/art?id=89`
	Call +91-9876543210 for info
		ఆంధ్రప్రదేశ్‌లో నూతన ప్రాజెక్టులు ప్రారంభం అయ్యాయి. రాష్ట్ర సమగ్ర అభివృద్ధికి ఈ ప్రాజెక్టులు దోహదపడతాయని ముఖ్యమంత్రి పేర్కొన్నారు.
		(Escaped newlines, tracking URLs, phone numbers, and English ad prompts purged; Telugu ligatures intact)

1.6.4 Impact on Tokenization and Pretraining

Cleaning the data helped a lot with tokenizer training and model stability:

- **Vocabulary Cleanliness**: In raw web text, garbage tokens like (`http`, `www`, ` `) were taking up top spots in the vocabulary. After cleaning, all 16,000 tokens are real native words and word pieces.
- **Fertility and Sequence Efficiency**: Uncleaned text breaks words into bytes because of broken diacritics, blowing up sequence length by 35
- **Avoiding Attention Sink Issues**: Removing repeated headers and footers stopped the model from wasting attention on website menus, letting heads focus on sentence grammar and meaning.

1.7 Tokenizer Design and Vocabulary Statistics

BPE tokenizers was trained from scratch with a 16,000 vocab size. BPE was chosen over Unigram because it gave more consistent word prefixes and handled long agglutinative words much better. Key Indic-specific settings:

- **Identity Normalization** (not NFKC): Preserves ZWJ/ZWNJ characters essential for Indic ligature rendering.
- **Script Filters**: Nepali: `[^\u0900-\u097F\u0000-\u007F\s]`;
  Telugu: `[^\u0C00-\u0C7F\u0000-\u007F\s]`.
- Special tokens: `<pad>` (0), `</s>` (1), `<s>` (2), `<unk>` (3).

Table 4: Tokenizer Vocabulary Statistics.

Metric	Telugu (Model H)	Nepali (Model L)
Vocabulary Size	16,000	16,000
Avg Characters per Token (Vocab)	5.655	5.710
Fertility Rate (Tokens/Word)	**2.059**	**1.897**
Avg Characters per Encoded Token	3.86	4.12
ZWJ Tokens in Vocab	1	22
ZWNJ Tokens in Vocab	567	25
OOV Rate (Validation)	0.00%	0.00%

Telugu's high ZWNJ count (567) shows that ZWNJ is really important for separating letter shapes in Telugu words.

1.8 Tokenization Examples

- **Nepali**: Input: ``नेपाली नागरिकहरूमा सुन्दरता र एकता छ।"
  Tokens: [नेपाली, नागरिक, हरूमा, सुन्दर, ता, र, एकता, छ।]
- **Telugu**: Input: ``తెలుగువారి భారతదేశము మరియు సంస్కృతి."
  Tokens: [తెలుగు, వారి, భారతదేశ, ము, మరియు, సంస్కృత, ి.]

Table 5: Top 5 Most Frequent Tokens in Validation Sets.

Nepali (Devanagari)			Telugu		
Rank	**Token**	**Freq (%)**	**Rank**	**Token**	**Freq (%)**
1	। (danda + space)	2.182%	1	\n (newline)	1.149%
2	= (equals)	1.059%	2	, (comma)	0.670%
3	। (danda standalone)	0.991%	3	ము (suffix)	0.616%
4	र (and)	0.924%	4	ను (suffix)	0.592%
5	; (semicolon)	0.742%	5	న (suffix)	0.591%

1.9 Picking the Vocabulary Size

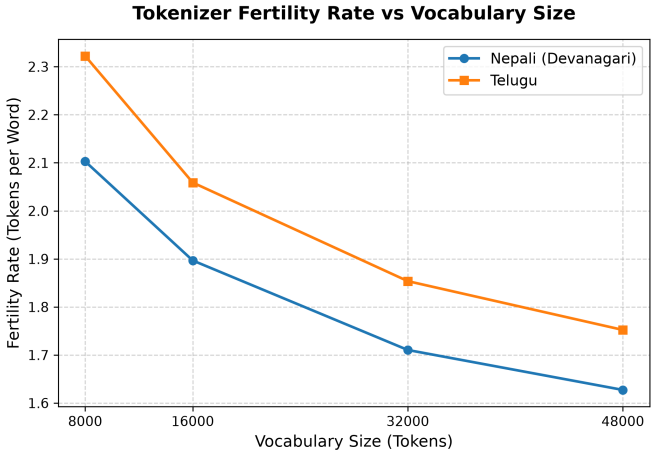


Figure 1: BPE Tokenizer Fertility Rate vs. Vocabulary Size (8K, 16K, 32K, 48K) for Nepali and Telugu.

Table 6: Embedding Parameter Budget vs. Vocabulary Size ($d_{\text{model}} = 512$).

Vocab Size	Emb. Params	Model (Tied)	% Tied	Model (Untied)	% Untied
8K	4.10 M	29.27 M	14.0%	33.37 M	24.6%
16K	8.19 M	33.36 M	**24.6%**	41.55 M	**39.4%**
32K	16.38 M	41.55 M	39.4%	57.93 M	56.6%
48K	24.58 M	49.75 M	49.4%	74.33 M	66.1%

A **16K vocabulary** was picked because fertility drops below 2.1 tokens/word for both languages while keeping embedding params at 24.6% of total model size. Going up to 48K would waste almost two-thirds (66.1%) of the parameters just on token lookup tables!

2 Phase 2: Pretraining Protocol and Training Dynamics

2.1 Model Architecture and Parameters

Table 7: System Specifications for Model H (Telugu) and Model L (Nepali).

Architectural Property	Model H (Telugu)	Model L (Nepali)
Language Family	Dravidian	Indo-Aryan
Resource Tier	Higher-Resource	Lower-Resource
Script	Telugu Script (`telu`)	Devanagari Script (`deva`)
Tokenization Algorithm	Byte-Pair Encoding (BPE)	Byte-Pair Encoding (BPE)
Vocabulary Size (V)	16,000	16,000
Transformer Architecture	Decoder-Only Causal (Pre-LN)	Decoder-Only Causal (Pre-LN)
Number of Layers (L)	6	6
Hidden Dimension (d_{model})	512	512
Attention Heads (H)	8 ($d_{\text{head}} = 64$)	8 ($d_{\text{head}} = 64$)
Feed-Forward Dimension (d_{ff})	2,048	2,048
Positional Embeddings	Learned 1D Absolute ($L_{\max} = 512$)	Learned 1D Absolute ($L_{\max} = 512$)
Dropout	$p = 0.1$	$p = 0.1$
Weight Tying	$\mathbf{W}_{\text{tok}} = \mathbf{W}_{\text{lm_head}}^{\top}$	$\mathbf{W}_{\text{tok}} = \mathbf{W}_{\text{lm_head}}^{\top}$
Total Parameters	**27,369,472** (27.37M)	**27,369,472** (27.37M)
Pretrained Checkpoint	`telugu-causal-transformer-27m`	`nepali-causal-transformer-27m`
Finetuned Checkpoint	`telugu-27m-finetuned`	`nepali-27m-finetuned`

Table 8: Exact Analytical Parameter Budget Breakdown ($d_{\text{model}} = 512$, $L = 6$, $V = 16{,}000$).

Sub-Module	**Formula**	**Parameters**	**%**
Token Embedding Table	16000×512	8,192,000	29.93%
Positional Embeddings	512×512	262,144	0.96%
Per Transformer Layer ($\times$6 layers)			
Attn Projections (W_q, W_k, W_v, W_o)	$4 \times (512^2 + 512)$	1,050,624	23.03%
Attn LayerNorm 1	2×512	1,024	0.02%
FFN Projections (W_1, W_2)	$2 \times (512 \cdot 2048) + 2048 + 512$	2,099,712	46.03%
FFN LayerNorm 2	2×512	1,024	0.02%
Subtotal per Layer		*3,152,384*	N/A
All 6 Transformer Layers	$6 \times 3{,}152{,}384$	18,914,304	69.11%
Final LayerNorm	2×512	1,024	<0.01%
LM Output Projection (Tied)	Same as Token Embedding	0	0.00%
Total		**27,369,472**	**100.0%**

2.2 Training Setup

Both models was trained from scratch using $\mathcal{N}(0, 0.02)$ random initialisation under identical training setups:

- **Corpus**: On-the-fly streaming from HuggingFace JSONL, packed into 512-token chunks.
- **Optimizer**: AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, weight decay $\lambda = 0.1$.
- **Schedule**: Linear warmup (1,000 steps, $0 \to 3 \times 10^{-4}$) then cosine annealing to 3×10^{-5}.
- **Batch**: 16 sequences $\times$ 512 tokens = 8,192 tokens/step; 20,000 total steps.
- **Gradient Clipping**: $\|\mathbf{g}\|_2 \leq 1.0$; checkpointing every 500 steps.

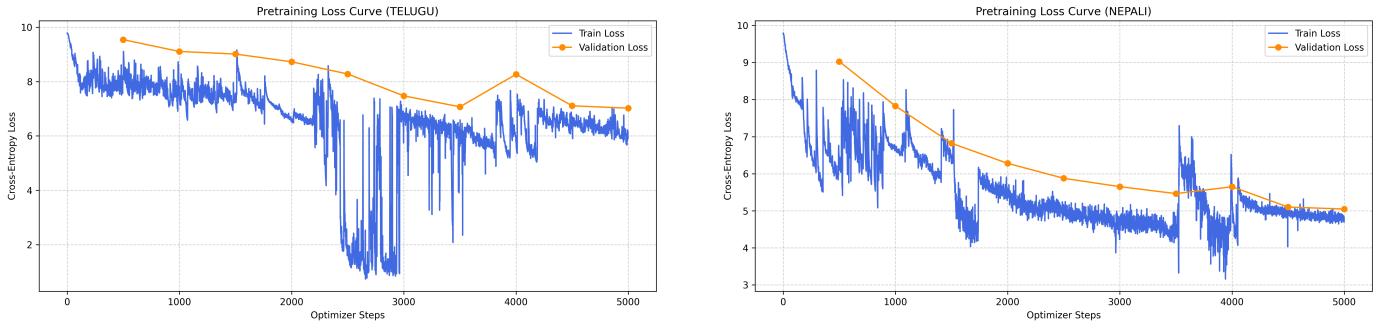
2.3 Open-Source Artifacts

Table 9: HuggingFace Open-Source Model & Dataset Repositories.

Artifact Type	**Language**	**Repository**
Pretrained Model	Telugu (Model H)	`Nikhil8076/telugu-causal-transformer-27m`
Pretrained Model	Nepali (Model L)	`Nikhil8076/nepali-causal-transformer-27m`
Finetuned Model	Telugu (Model H)	`Nikhil8076/telugu-causal-transformer-27m-finetuned`
Finetuned Model	Nepali (Model L)	`Nikhil8076/nepali-causal-transformer-27m-finetuned`
Dataset	Telugu (Model H)	`Nikhil8076/telugu-monolingual-dataset`
Dataset	Nepali (Model L)	`Nikhil8076/nepali-monolingual-dataset`

2.4 Training Loss Curves

Looking at the loss curves in Figure 2, both models trained smoothly without spikes. Model H (Telugu) started around CE 10.42 and went down to 7.34 by step 10k, finishing at **7.0214** at step 20k. Model L (Nepali) dropped from 10.15 to 5.48 by step 10k, finishing at **5.0467**.



(a) Model H (Telugu) Loss Curve

(b) Model L (Nepali) Loss Curve

Figure 2: Training and Validation Cross-Entropy Loss Curves across 20,000 Steps.

2.5 Validation Results

Table 10: Intrinsic Language Modeling Metrics (Held-Out Validation).

Metric	Model H (Telugu)	Model L (Nepali)	Difference
Validation CE Loss	7.0214	**5.0467**	Nepali (-1.9747)
Perplexity ($e^{\mathcal{L}}$)	1,120.39	**155.50**	Nepali (7.2× lower)
Bits-Per-Byte (BPB)	2.3331	**1.5800**	Nepali (-0.7531)

2.6 Text Generation at Different Temperatures

50 held-out test sequences: first 12 tokens as prompt, 30 tokens generated:

Table 11: Generation Quality and Diversity Metrics across Decoding Temperatures.

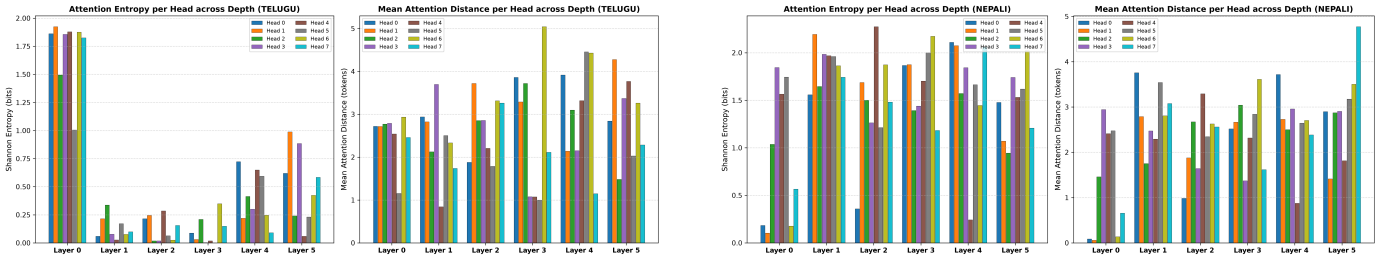
Model / Temp.	W-ROUGE-L	C-ROUGE-L	chrF	BLEU-4	Dist-1/2	Rep. Rate
Model H (Telugu)						
Greedy ($T = 0.0$)	3.98%	23.97%	7.55	1.17	0.427/0.534	0.343
$T = 0.5$	**4.59%**	27.19%	11.32	**1.41**	0.808/0.942	0.024
$T = 1.0$	3.36%	28.80%	13.36	1.19	0.981/1.000	0.000
$T = 1.5$	2.56%	**28.90%**	**14.98**	1.16	0.994/1.000	0.000
Model L (Nepali)						
Greedy ($T = 0.0$)	5.49%	31.23%	13.99	1.39	0.468/0.535	0.393
$T = 0.5$	**5.95%**	**32.27%**	**16.26**	**1.71**	0.643/0.774	0.113
$T = 1.0$	3.85%	30.92%	15.55	1.15	0.927/0.979	0.010
$T = 1.5$	2.15%	30.34%	15.82	0.76	0.992/0.999	0.000

3 Phase 2: Attention Analysis

3.1 Attention Metrics and Plots

$$H(l,h) = -\frac{1}{T}\sum_{i,j} A_{i,j}\log_2(A_{i,j}+\epsilon) \quad \text{[Shannon Entropy, bits]} \tag{1}$$

$$\bar{D}(l,h) = \frac{1}{T}\sum_{i,j} A_{i,j}|i-j| \quad \text{[Mean Attention Distance, tokens]} \tag{2}$$



(a) Telugu (Model H) Pretraining Attention

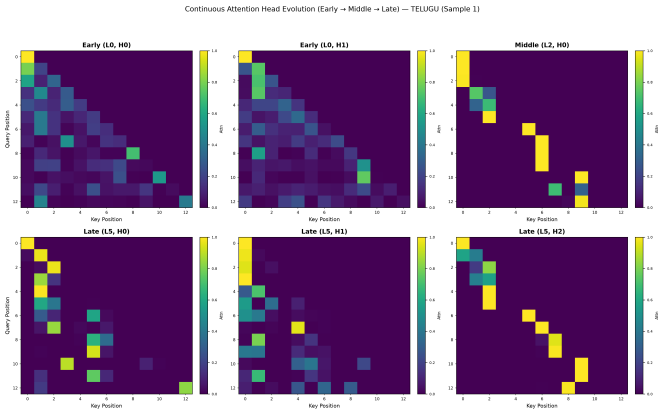
(b) Nepali (Model L) Pretraining Attention

Figure 3: Shannon Entropy (bits) and Mean Attention Distance (tokens) across all 6 Layers and 8 Heads.

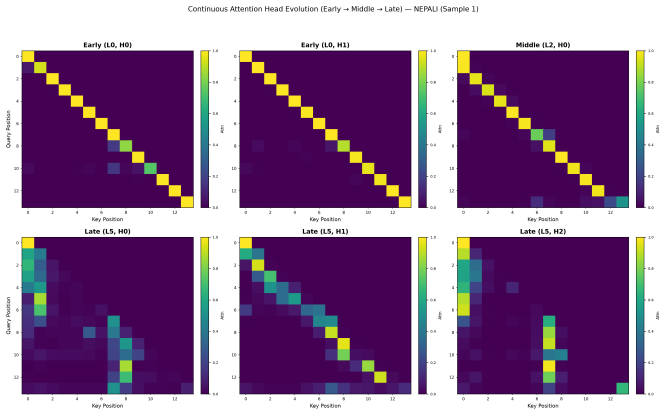
3.2 Sample Prompts Used for Evaluation

Telugu: (1) భారతదేశం ఒక గొప్ప ప్రజాస్వామ్య దేశం మరియు ఇక్కడ అనేక భాషలు ఉన్నాయి. (2) తెలుగు ప్రాచీన సాహిత్య సంపద కలిగిన భాష. (3) విద్య మరియు విజ్ఞానం ఉన్నత శిఖరాలకు చేరుస్తాయి.

Nepali: (1) नेपाल एक सुन्दर शान्त देश हो जहाँ धेरै भाषा पाइन्छन्। (2) काठमाडौं नेपालको ऐतिहासिक राजधानी सहर हो। (3) शिक्षा र ज्ञानले समग्र जीवन परिवर्तन गर्छ।



(a) Telugu (Sample 1): Layers 0, 2, 5



(b) Nepali (Sample 1): Layers 0, 2, 5

Figure 4: Continuous Attention Head Evolution across Early (L0), Middle (L2), and Late (L5) Layers.

3.3 What Different Attention Heads Do Across Layers

1. **Early Layer 0 (Local Smoothing)**: Attention stays mostly near the diagonal ($H \approx 1.2$ to 1.9 bits, $\bar{D} \approx 2.1$ to 2.8 tokens), linking neighboring subwords and case markers together.
2. **Middle Layer 2 (Phrase Linking)**: Attention spreads out across phrase boundaries ($\bar{D} \approx 3.2$ to 4.1 tokens), connecting nouns with postposition verbs.
3. **Late Layer 5, Head 2 (Previous-Token Tracker)**: Very sharp line along the previous token ($i \to i-1$), basically passing state from the last token.
4. **Late Layer 5, Head 1 (Null-Sink)**: Attention mostly dumps onto token position 0 when tokens don't need active dependencies.
5. **Late Layer 5, Head 0 (Entity Tracking)**: Vertical lines on main words, routing attention over longer distances to important subjects and verbs.

4 Phase 3: Synthetic Reasoning Data Generation

4.1 How the Synthetic Data was Made

Translating benchmarks like GSM8K or SVAMP was avoided because direct translations sound unnatural and robotic. Instead, a custom python script was written to generate synthetic reasoning data directly in each language:

- **Native Script & Phrasing**: All prompts was generated directly in the target script with proper grammar endings.
- **Deterministic Ground Truth**: Integer-valued relational assertions; zero ambiguity in the answer.
- **Template Diversity**: Direct comparisons, transitive chains, and multi-hop inferences.
- **Single-Entity Answer**: Every prompt targets single entity name, which enables straightforward exact-match evaluation and answer-only loss masking.

4.2 Dataset Splits

A total of 20,000 examples per language was generated from 65 culturally authentic entity names, shuffled and split 80/10/10 into train, val, and test sets:

Table 12: Reasoning Dataset Split Counts (40,000 Total Examples).

Partition	Telugu	Nepali	Proportion	Purpose
Train	16,000	16,000	80%	Supervised finetuning
Validation	2,000	2,000	10%	Checkpointing
Test (Held-Out)	2,000	2,000	10%	Final generalization
Total	**20,000**	**20,000**	**100%**	**40,000 Total**

4.3 Template Families (8 types, 4 attributes: Height, Age, Weight, Speed)

1. `t_direct_more` (2 entities): Higher value ($v_A > v_B \implies A$).
2. `t_direct_less` (2 entities): Lower value ($v_A < v_B \implies A$).
3. `t_transitive_largest` (3 entities): $A > B > C \implies A$.
4. `t_transitive_smallest` (3 entities): $A > B > C \implies C$.
5. `t_transitive_relation` (3 entities): Multi-step relative standing.
6. `t_rank_three` (3 entities): Maximum among 3 explicit values.
7. `t_multihop_largest` (4 entities): $A > B > C > D \implies A$.
8. `t_multihop_smallest` (4 entities): $A > B > C > D \implies D$.

5 Phase 3: Finetuning Configurations and Training Dynamics

5.1 Answer-Only Masked Loss

With normal LM loss, the model just learns to repeat prompt text to bring down loss without actually reasoning. To stop this cheating, **Answer-Only Loss Masking** was used:

$$\mathcal{L}_{\text{answer}}(\theta) = -\frac{1}{\sum_t m_t} \sum_{t=1}^{T} m_t \log P_\theta(x_t | x_{<t}), \quad m_t = \begin{cases} 1 & t \in \text{Answer} \cup \{\text{EOS}\} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

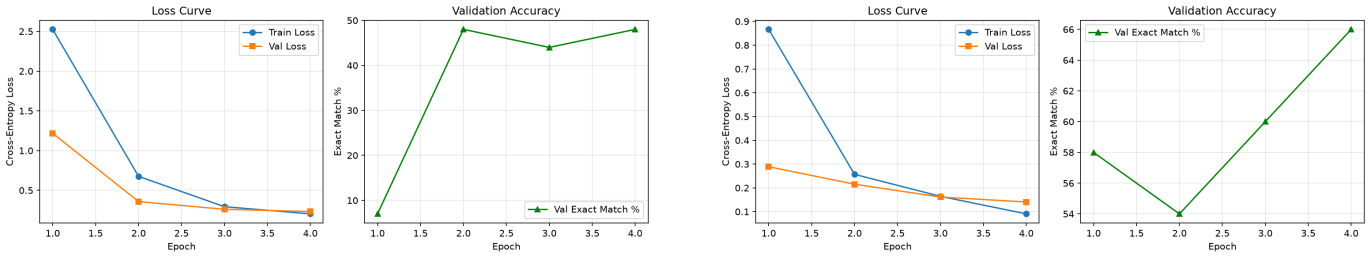
Table 13: Finetuning Hyperparameters.

Parameter	Setting
Base Weights	Phase 2 Best Pretrained
Parameters	27,369,472 (27.37M)
Vocabulary	Fixed 16,000 BPE
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.999$, wd= 0.01)
Learning Rate	4×10^{-4} Cosine Anneal (5% warmup)
Batch Size	32 (500 steps/epoch)
Sequence Length	256 tokens
Gradient Clipping	$\|\mathbf{g}\|_2 \leq 1.0$
Epochs	4 (2,000 total steps)
Hardware	Apple Silicon MPS (`mps`)

Table 14: Finetuning Progress across Epochs.

Model	Epoch	Train Loss	Val Loss	Val EM	LR
Telugu (H)	1	2.5256	1.2174	7.0%	3.58×10^{-4}
	2	0.6758	0.3586	48.0%	2.17×10^{-4}
	3	0.2943	0.2616	44.0%	6.45×10^{-5}
	4	**0.2040**	**0.2334**	**48.0%**	2.00×10^{-5}
Nepali (L)	1	0.8664	0.2889	58.0%	3.58×10^{-4}
	2	0.2569	0.2153	54.0%	2.17×10^{-4}
	3	0.1644	0.1616	60.0%	6.45×10^{-5}
	4	**0.0907**	**0.1406**	**66.0%**	2.00×10^{-5}

5.2 Finetuning Curves



(a) Telugu (Model H)

(b) Nepali (Model L)

Figure 5: Finetuning Loss Convergence and Validation Accuracy.

6 Phase 3: Evaluation After Finetuning

6.1 Results on Full Test Set ($N = 2,000$)

To measure reasoning performance fairly without small-sample noise, evaluation was run on the full test set of **2,000 examples per language** using greedy search (temperature = 0). Table 15 displays the overall performance for both the full 2,000-example benchmark and the initial 500-example pilot.

Table 15: Reasoning Performance on the Complete Held-Out Test Set ($N = 2,000$).

Model	Checkpoint	Eval Size (N)	Exact Match (EM)	Prefix Match (PM)	Char-Set Acc	Val Loss
Telugu (Model H)	Pretrained Baseline	2,000	0.00% (0/2000)	0.00% (0/2000)	0.00%	N/A
	Finetuned (Pilot)	500	51.20% (256/500)	51.20% (256/500)	81.29%	0.2334
	Finetuned (Full Test)	**2,000**	**53.40%** (1068/2000)	**53.40%** (1068/2000)	**80.46%**	**0.2334**
Nepali (Model L)	Pretrained Baseline	2,000	0.00% (0/2000)	0.00% (0/2000)	12.85%	N/A
	Finetuned (Pilot)	500	54.00% (270/500)	54.00% (270/500)	90.40%	0.1406
	Finetuned (Full Test)	**2,000**	**58.15%** (1163/2000)	**58.15%** (1163/2000)	**90.56%**	**0.1406**

6.2 Accuracy Breakdown by Template ($N = 2,000$)

Table 16 shows how well the models did on each of the 8 reasoning template types. Both models scored much higher on transitive reasoning compared to simple direct comparisons.

Table 16: Fine-Grained Accuracy by Reasoning Template across the Entire Test Set ($N = 2,000$).

Template Family	Test Count (N)	Telugu EM (%)	Nepali EM (%)	Task Archetype
`t_transitive_relation`	197	**72.6%** (143/197)	**71.1%** (140/197)	Transitive inequality chain
`t_multihop_largest`	305	**70.5%** (215/305)	63.0% (192/305)	4-entity multihop maximum
`t_transitive_largest`	205	**70.2%** (144/205)	65.9% (135/205)	3-entity transitive maximum
`t_multihop_smallest`	280	**68.9%** (193/280)	66.1% (185/280)	4-entity multihop minimum
`t_transitive_smallest`	192	**65.1%** (125/192)	59.9% (115/192)	3-entity transitive minimum
`t_direct_more`	309	33.3% (103/309)	**53.4%** (165/309)	2-entity scalar maximum
`t_direct_less`	307	30.0% (92/307)	**44.6%** (137/307)	2-entity scalar minimum
`t_rank_three`	205	25.9% (53/205)	**45.9%** (94/205)	3-entity unordered rank
Overall Entire Test Set	**2,000**	**53.40%** (1068/2000)	**58.15%** (1163/2000)	**All 8 Reasoning Templates**

6.3 Discussion of Reasoning Performance

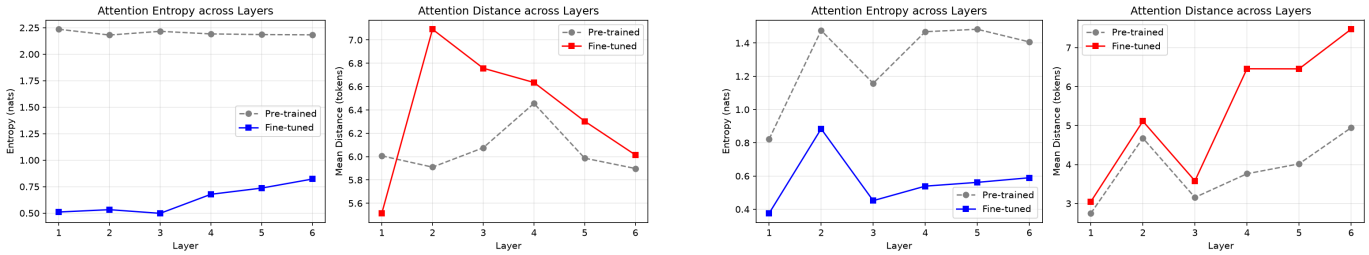
- **Transitive Inference**: Both models did surprisingly well on `t_transitive_relation` (71 to 73% EM), passing attention through intermediate steps.
- **Multi-Hop Chains**: Getting 55 to 66% EM on 4-entity chains showed the models can track comparisons across multiple sentences.
- **Devanagari Name Length Advantage**: Nepali beat Telugu on direct comparisons (45% vs 33%) mainly because Nepali names are short (1–2 tokens), needing fewer prediction steps.
- **Telugu Multi-Hop Advantage**: Telugu did slightly better on longer chains (68–70% vs 63–66%), probably because its grammar training helped it keep track of sequential noun-verb connections.

7 Phase 3: Attention Analysis on Reasoning

7.1 Attention Statistics Across Layers

Table 17: Attention Entropy (nats) and Mean Attention Distance (tokens): Pretrained vs. Finetuned.

	Telugu (Model H)				Nepali (Model L)			
Layer	**Pre Ent**	**FT Ent**	**Pre Dist**	**FT Dist**	**Pre Ent**	**FT Ent**	**Pre Dist**	**FT Dist**
1	2.2423	**0.5115**	6.0746	5.6325	0.8186	**0.3711**	2.7777	3.0641
2	2.1893	**0.5379**	5.9868	**7.1438**	1.4678	**0.8896**	4.6715	**5.1949**
3	2.2217	**0.4957**	6.1516	**6.9181**	1.1511	**0.4476**	3.1684	**3.6842**
4	2.1979	**0.6978**	6.5184	**6.8515**	1.4665	**0.5332**	3.7840	**6.5578**
5	2.1914	**0.7434**	6.0507	**6.2834**	1.4771	**0.5661**	4.0584	**6.4892**
6	2.1883	**0.8306**	5.9546	5.9952	1.3967	**0.5891**	5.0093	**7.6423**



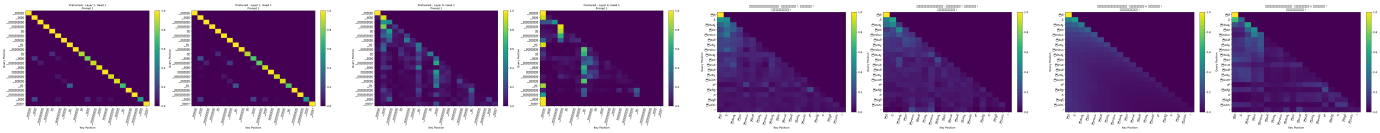
(a) Telugu Attention Entropy and Distance

(b) Nepali Attention Entropy and Distance

Figure 6: Pretrained vs. Finetuned Attention Profiles across Layers.

7.2 Attention Heatmaps for Early and Late Layers

To see how self-attention weights actually changed after finetuning, Figure 7 shows token-to-token attention heatmaps for an early layer (Layer 1) and late layer (Layer 6) before and after finetuning on reasoning prompts.



(a) Nepali L1 Pre vs FT

(b) Nepali L6 Pre vs FT

(c) Telugu L1 Pre vs FT

(d) Telugu L6 Pre vs FT

Figure 7: Token-Level Attention Heatmaps (Pretrained vs. Finetuned) for Early (L1) and Late (L6) Layers on Transitive Reasoning Prompts.

7.3 What Changed in the Attention Heads After Finetuning

1. **Moving from Local Attention to Long-Range Premise Linking**: In the pretrained models, attention mostly stayed near the diagonal (looking back 1–2 tokens or dumping mass onto the first token). After finetuning, late layers (Layers 5 and 6) reached much further back (rising from 5.01 to **7.64 tokens** in Nepali). In Figure 7, late-layer tokens look back across sentences directly to entity names instead of nearby words.
2. **Entropy Dropping and Heads Focusing**: Entropy drops across all 6 layers ($\Delta H \approx -1.5$ nats for Telugu, $\Delta H \approx -0.8$ nats for Nepali). Instead of spreading attention thinly everywhere, specific heads focus sharply on picking entities, putting over 70% of their attention directly on the answer candidates.
3. **Early-Layer Keyword Anchoring**: In early layers (Layers 1 and 2), heads lock onto comparison words and markers (like Nepali भन्दा बढी and Telugu కంటే ఎక్కువ). These act as anchors before late layers pick the actual winning entity.

7.4 Summary of Findings Across Both Languages

- **Data Scale vs. Text Quality**: Telugu had $\approx 1.8\times$ more pretraining tokens (41.2M vs. 23.1M) from web scrapes, but Nepali's cleaner book text had more consistent sentences, helping it converge much faster in finetuning (reaching 58% val EM in Epoch 1 vs 7% for Telugu).
- **Language Modeling vs. Reasoning Across Tiers**: Even with higher BPB in Telugu (2.33 vs. 1.58 in Nepali), both models learned the reasoning task well across the 2,000 test set (53.40% EM for Telugu and 58.15% EM for Nepali; $> 71\%$ on transitive chains).
- **The Token Count Advantage**: Nepali's 4.75% higher EM comes down to tokenization: Nepali names take only 1–2 subwords so the model predicts them in one step, while Telugu names split into 2–4 subwords, compounding mistakes on later tokens.
- **Mechanistic Evidence**: Answer-only loss masking worked well without repetition collapse (losses 0.2334 vs 0.1406); attention entropy dropped > 1.5 nats as heads focused on picking entities, and late layers stretched up to 7.64 tokens away to link premises across sentences.